Requirement-to-Architecture Alignment via Knowledge-Guided Multi-Agent Reasoning

Anonymous Author(s)

Abstract

Natural-language requirements often describe expected system behavior without making the architectural implications explicit. This paper studies the problem of aligning requirements with software architecture through a knowledge-guided multi-agent reasoning workflow. Rather than proposing a general platform, we focus on how analyst, architecture, and reviewer agents can extract architectural concerns, map requirements to components and constraints, and maintain requirement-to-architecture traceability under human supervision. The planned evaluation examines whether the workflow improves architecture alignment accuracy, mapping completeness, traceability quality, and architectural consistency compared with non-architecture-aware baselines.

Keywords

large language models, multi-agent systems, software engineering, requirements engineering, software architecture, requirement-to-architecture mapping, traceability

1 Introduction

1.1 Background and Motivation

Natural-language requirements are the main interface between stakeholder intent and software design. However, requirements are often written without explicit architectural assumptions, component responsibilities, quality-attribute trade-offs, or design constraints. This gap makes it difficult for intelligent software systems to move from requirement understanding to architecture-aware design decisions. Recent requirements engineering research has also emphasized the central role of requirements in intelligent software development [2] and the persistent difficulty of writing precise yet stakeholder-readable requirement specifications [1].

1.2 Challenges of Requirement-to-Architecture Alignment

This paper focuses on the requirement-to-architecture alignment problem. The key challenges are: identifying architectural concerns from informal requirements, mapping requirements to components and responsibilities, aligning quality attributes with architecture constraints, and preserving traceability between requirement statements and design decisions.

1.3 Multi-Agent Architecture Reasoning

We investigate a lightweight multi-agent reasoning workflow for architecture-aware requirements engineering. The workflow separates semantic requirement analysis, architecture reasoning, and review-oriented consistency checking, while keeping human feedback in the loop for ambiguous or high-impact mapping decisions.

1.4 Contributions

The paper is structured around the following research contributions:

- A clear formulation of requirement-to-architecture alignment for intelligent software systems.
- A knowledge-guided multi-agent workflow for extracting architectural concerns and constructing requirement-to-architecture mappings.
- A traceability-oriented mapping method covering component identification, responsibility allocation, and architecture constraint alignment.
- An empirical evaluation plan for assessing alignment accuracy, mapping completeness, traceability quality, and architecture consistency.

2 Related Work

2.1 Requirement-to-Architecture Mapping

This subsection positions the paper against approaches that connect requirement statements with architectural elements, design responsibilities, interfaces, and quality-attribute decisions. The discussion should emphasize why architecture-aware mapping is different from general requirement extraction.

2.2 Architecture Recovery and Software Architecture Reasoning

This subsection reviews work on recovering, analyzing, and reasoning about software architecture. The focus is on architectural views, components, connectors, constraints, and design rationale that can support requirement-to-architecture alignment.

2.3 Requirement Traceability

This subsection covers trace links between requirements and downstream design artifacts. The discussion should highlight traceability quality, link completeness, and the role of trace links in validating architectural consistency.

2.4 LLM-Based Requirement Engineering

This subsection discusses LLM-based requirement engineering only where it is relevant to architecture awareness. The focus is on requirement understanding, quality-attribute extraction, and design implication analysis, rather than general LLM capabilities.

2.5 Multi-Agent Collaboration in Software Engineering

This subsection reviews multi-agent collaboration patterns for software engineering tasks. The discussion should be limited to role separation, cross-checking, and human-supervised reasoning that support requirement-to-architecture mapping.

3 Methodology

3.1 Framework Overview

The proposed method takes natural-language requirements as input and produces an architecture-aware alignment artifact. The artifact contains architectural concerns, candidate components, responsibility assignments, architecture constraints, and trace links from requirements to architectural decisions.

3.2 Multi-Agent Architecture Alignment Workflow

The workflow uses three coordinated roles rather than a large agent hierarchy. Each role operates on the same requirement set and exchanges structured intermediate artifacts for review.

3.2.1 Analyst Agent. The analyst agent extracts functional requirements, quality attributes, constraints, domain terms, and architecturally significant concerns from natural-language requirements.

3.2.2 Architecture Agent. The architecture agent proposes candidate components, connectors, responsibilities, and design constraints based on the extracted architectural concerns and the available architecture knowledge.

3.2.3 Reviewer Agent. The reviewer agent checks whether the proposed mapping is complete, consistent, and traceable. It identifies unmapped requirements, weak responsibility assignments, and architecture decisions that lack requirement evidence.

3.3 Requirement Semantic Analysis

This step structures requirements into architecture-relevant categories.

3.3.1 Functional Requirements. Functional requirements are analyzed to identify system capabilities, user-facing behaviors, and candidate responsibilities.

3.3.2 Constraints. Constraints are extracted as limitations on technology, integration, compliance, deployment, data handling, or operational behavior.

3.3.3 Quality Attributes. Quality attributes such as performance, availability, scalability, maintainability, security, and usability are represented as architecture-driving concerns.

3.3.4 Architectural Concerns. Architectural concerns summarize the design implications that should influence component boundaries, communication patterns, and architecture constraints.

3.4 Knowledge-Guided Architecture Reasoning

Architecture reasoning is guided by reusable knowledge rather than relying only on free-form generation.

3.4.1 Architecture Knowledge. The knowledge source contains architecture concepts, common component types, connector semantics, quality-attribute tactics, and design-decision criteria.

3.4.2 Design Patterns. Design patterns are used as reasoning aids when requirements imply recurring responsibility structures or interaction styles.

3.4.3 Architecture Templates. Architecture templates provide lightweight reference structures for common system families and help standardize mapping outputs.

3.4.4 Prompt Strategies. Prompt strategies constrain the agents to produce structured mappings, cite requirement evidence, and separate assumptions from supported architecture decisions.

3.5 Requirement-to-Architecture Mapping

This step is the central output of the method. It links requirement-level evidence to architectural elements and design constraints.

3.5.1 Component Identification. Candidate components are identified from functional responsibilities, domain concepts, external actors, data ownership, and quality-attribute drivers.

3.5.2 Responsibility Allocation. Responsibilities are assigned to components with explicit references to the requirement statements that justify each assignment.

3.5.3 Architecture Constraint Alignment. Architecture constraints are checked against extracted requirement constraints and quality attributes to reduce requirement-design inconsistency.

3.5.4 Traceability Construction. Trace links connect requirements, architectural concerns, components, responsibilities, constraints, and reviewer comments.

3.6 Human-in-the-Loop Interaction

Human reviewers validate ambiguous mappings, approve assumptions with architectural impact, and revise agent outputs when requirement evidence is insufficient. The interaction is limited to requirement-to-architecture alignment decisions.

4 Experiments

4.1 Research Questions

The evaluation is organized around the following research questions:

- RQ1: How accurately does the workflow align natural-language requirements with architectural elements?
- RQ2: How complete and useful are the generated requirement-to-architecture trace links?
- RQ3: How much do architecture knowledge and agent review contribute to mapping quality?
- RQ4: How do human reviewers assess the usefulness of the alignment artifacts?

4.2 Experimental Setup

This subsection will describe the implementation setting, agent roles, knowledge sources, prompting protocol, human review procedure, and evaluation process. It should report configuration details without overstating tool generality.

4.3 Datasets

The dataset should contain requirement documents paired with reference or expert-reviewed architectural artifacts. The paper should describe project domains, requirement counts, artifact types, and annotation procedures once the dataset is finalized.

4.4 Baselines

Baselines should reflect requirement-to-architecture mapping alternatives, such as direct single-agent mapping, LLM-only mapping without architecture knowledge, and mapping without reviewer feedback.

4.5 Evaluation Metrics

4.5.1 Architecture Alignment Accuracy. Measures whether generated architectural elements and constraints match reference or expert-accepted mappings.

4.5.2 Requirement Traceability Quality. Measures the correctness, usefulness, and evidence support of requirement-to-architecture trace links.

4.5.3 Mapping Completeness. Measures whether architecturally significant requirements are covered by components, responsibilities, constraints, or trace links.

4.5.4 Architecture Consistency. Measures whether the produced architecture mapping avoids unsupported decisions, responsibility conflicts, and constraint violations.

4.6 Main Results

This subsection will report quantitative and qualitative findings after the evaluation is conducted. No result claims should be added before experimental evidence is available.

4.7 Ablation Study

The ablation study should isolate the contribution of architecture knowledge, role-separated agents, reviewer checks, and human feedback.

4.8 Case Study

The case study should illustrate one requirement-to-architecture mapping workflow, including requirement evidence, architectural concerns, mapped components, constraints, and reviewer feedback.

5 Threats to Validity

5.1 Internal Validity

Internal threats may arise from prompt sensitivity, agent interaction order, evaluator subjectivity, and the quality of human-reviewed reference mappings. The study should mitigate these threats through fixed protocols, independent review, and documented adjudication.

5.2 External Validity

External threats concern whether results generalize across domains, requirement styles, system sizes, and architecture paradigms. The evaluation should report dataset diversity and avoid claims beyond the studied project scope.

5.3 Construct Validity

Construct threats concern whether the selected metrics faithfully measure requirement-to-architecture alignment. The study should justify each metric and combine automatic assessment with expert review where necessary.

6 Conclusion

This paper targets requirement-to-architecture alignment as a research problem in architecture-aware requirement engineering. The proposed structure centers on a knowledge-guided multi-agent reasoning workflow that extracts architectural concerns, maps requirements to components and constraints, and constructs trace links under human supervision. The planned evaluation focuses on whether this workflow improves alignment accuracy, traceability quality, mapping completeness, and architecture consistency.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix may include supplementary materials such as the detailed annotation protocol, prompt templates, architecture knowledge schema, and additional case-study artifacts. These materials should support the main requirement-to-architecture alignment study without introducing new research scope.